



Image, sécurité et deep learning

Compte-rendu TP1

Chiffrement d'images

Louis Jean
Master 2 IMAGINE
Université de Montpellier
N° étudiant : 21914083

9 septembre 2024

Table des matières

1	Introduction	2
2	Chiffrement d'images par permutation	2
2.1	Implémentation du programme	2
2.2	Tests et résultats	2
2.3	Comparaison des images et des histogrammes	3
2.4	Déchiffrement de l'image	4
3	Chiffrement d'images par substitution	5
3.1	Implémentation du programme	5
3.2	Tests et résultats	5
3.3	Comparaison des images et des histogrammes	6
3.4	Déchiffrement de l'image	7
3.5	Attaque par force brute	7
4	Conclusion	8

1 Introduction

L'objectif de ce premier TP est de chiffrer des images et de faire la différence entre un chiffrement par permutation et un chiffrement par substitution.

2 Chiffrement d'images par permutation

Le chiffrement par permutation consiste à déplacer les pixels d'une image sans altérer leurs valeurs. Pour ce faire, j'ai développé un programme `permutation.cpp` qui prend en entrée une image en niveau de gris (format PGM), une clé secrète `K` de 8 bits, et produit une image chiffrée de même taille.

2.1 Implémentation du programme

Le programme `permutation.cpp` réalise le chiffrement d'une image en permutant ses pixels selon une clé secrète. Voici les étapes principales de l'algorithme :

- **Initialisation** : Un tableau `positions` de la taille de l'image est créé pour garder la trace des positions déjà utilisées dans l'image chiffrée. Toutes les positions sont initialisées à `-1`.
- **Génération de nombres aléatoires** : La fonction `srand(cle)` initialise le générateur de nombres aléatoires avec la clé secrète `cle`, assurant que la séquence de nombres aléatoires sera la même pour une même clé.
- **Permutation des pixels** : Pour chaque pixel de l'image originale, une position aléatoire `pos` est générée en utilisant `rand() % nTaille` (`nTaille` étant le nombre de pixels total dans l'image). Si cette position n'a pas encore été utilisée (vérifiée en consultant le tableau `positions`), le pixel est placé à cette position dans l'image chiffrée. Si la position est déjà utilisée, la position est incrémentée (avec retour à zéro si nécessaire) jusqu'à trouver une position libre.

2.2 Tests et résultats

J'ai testé le programme avec deux images, la première tirée du film *Alien Romulus* et la seconde issue du film *John Wick*, et différentes valeurs de clé.



(a) Image *Alien*



(b) Image *John Wick*

Figure 1: Images originales

Voici les résultats obtenus pour les images *Alien* et *John Wick* avec les clés 15 et 239. Le PSNR a été calculé entre l'image originale et l'image chiffrée à chaque fois.

Table 1: Résultats du chiffrement par permutation pour différentes clés

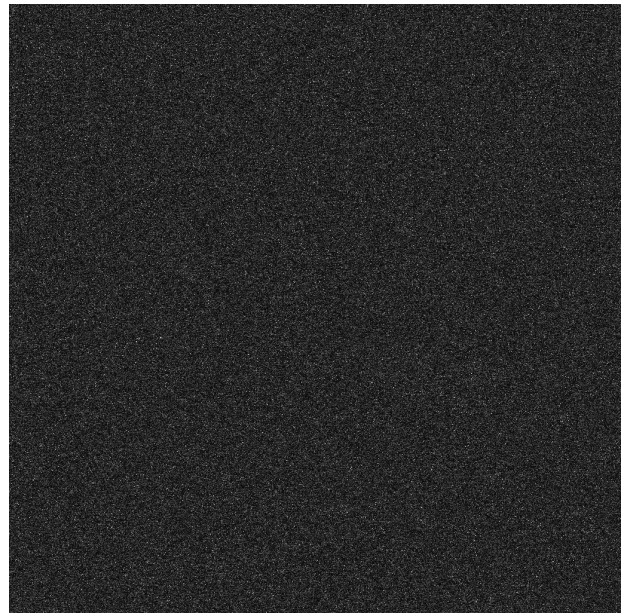
Image	Clé	PSNR (dB)	Entropie originale (bits/pixel)	Entropie chiffrée (bits/pixel)
Alien	15	14,8722	6,56509	6,56509
John Wick	15	9,9019	7,48395	7,48395
Alien	239	14,8898	6,56509	6,56509
John Wick	239	9,91987	7,48395	7,48395

On remarque que les valeurs d'entropie sont identiques pour l'image originale et l'image chiffrée, ce qui est cohérent avec le fait que le chiffrement par permutation ne modifie pas la distribution des niveaux de gris de l'image. On observe aussi que les valeurs de clés différentes produisent des permutations différentes donc des variations au niveau du PSNR, qui est sensible à la position des pixels.

2.3 Comparaison des images et des histogrammes



(a) Image originale *Alien*



(b) Image chiffrée par permutation (clé 15)

Figure 2: Comparaison des images originales et chiffrées

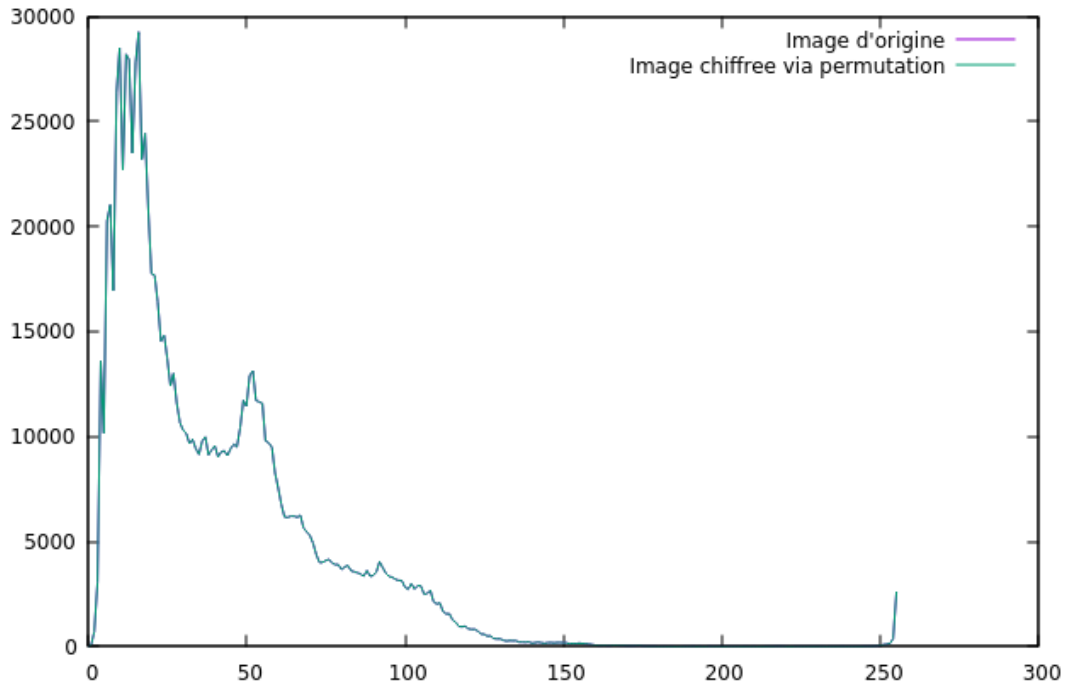


Figure 3: Histogrammes de l'image originale *Alien* et de l'image chiffrée par permutation

Comme on peut le voir sur les histogrammes, le chiffrement par permutation n'affecte pas la distribution des niveaux de gris de l'image, ce qui se traduit par des histogrammes identiques pour l'image originale et l'image chiffrée.

2.4 Déchiffrement de l'image

J'ai également développé un programme `permut_inv.cpp` permettant de déchiffrer une image chiffrée avec la même clé. Il repose essentiellement sur les opérations inverses du chiffrement par permutation. Il est important de noter qu'aucune perte n'est introduite lors du déchiffrement.

3 Chiffrement d'images par substitution

Le chiffrement par substitution consiste à remplacer les valeurs des pixels de l'image par d'autres valeurs selon une certaine règle dépendant de la clé.

3.1 Implémentation du programme

Le programme `substitution.cpp` réalise le chiffrement d'une image en substituant les valeurs des pixels selon un algorithme basé sur une clé secrète. Voici les étapes principales de l'algorithme :

- **Initialisation** : Le générateur de nombres aléatoires est initialisé avec la clé secrète `cle` en utilisant `srand(cle)`.
- **Substitution des pixels** : Le chiffrement est réalisé en parcourant chaque pixel de l'image originale et en calculant la valeur correspondante dans l'image chiffrée. Le processus est le suivant :
 1. Pour le premier pixel ($i = 0$), un nombre aléatoire k est généré entre 0 et 255. La valeur du pixel chiffré est alors calculée par :

$$\text{ImgOut}[0] = (\text{ImgIn}[0] + k) \mod 256$$

2. Pour chaque pixel suivant ($i \geq 1$), un nouveau nombre aléatoire k est généré, et la valeur du pixel chiffré est calculée en additionnant la valeur du pixel chiffré précédent, la valeur du pixel original courant, et k , le tout modulo 256 :

$$\text{ImgOut}[i] = (\text{ImgOut}[i-1] + \text{ImgIn}[i] + k) \mod 256$$

3.2 Tests et résultats

En utilisant la clé 239 sur l'image *Alien*, j'ai obtenu les résultats suivants :

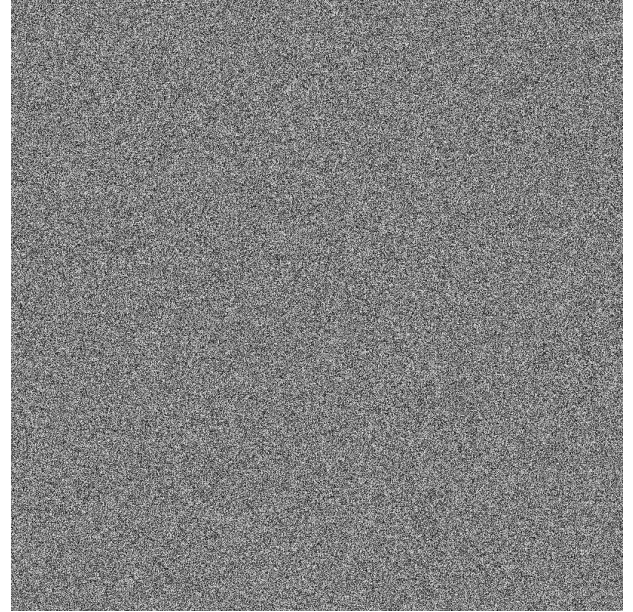
- PSNR : 6,62839 dB
- Entropie de l'image originale : 6,56509 bits/pixel
- Entropie de l'image chiffrée : 7,99979 bits/pixel

Ici, on remarque que l'entropie de l'image change lors du chiffrement, car on modifie directement les valeurs des pixels. Cela nous permettra plus tard de réaliser une attaque sur cette méthode de chiffrement.

3.3 Comparaison des images et des histogrammes



(a) Image originale *Alien*



(b) Image chiffrée par substitution (clé 239)

Figure 4: Comparaison des images originales et chiffrées par substitution

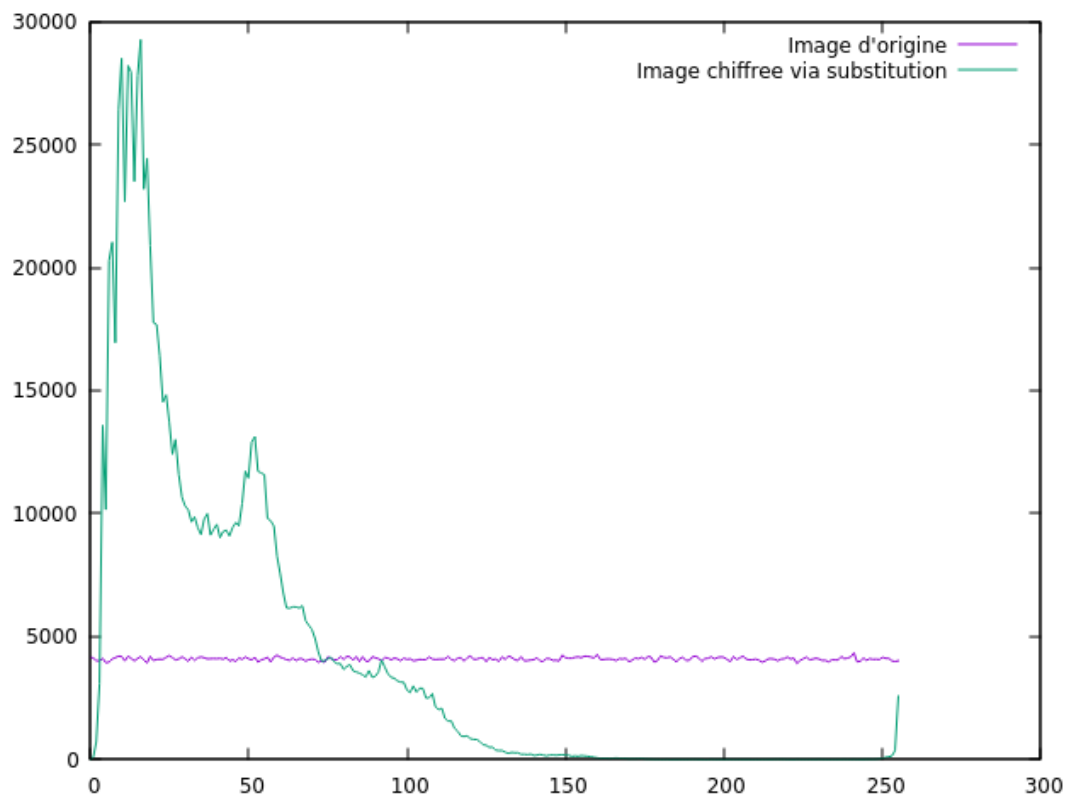


Figure 5: Histogrammes de l'image originale *Alien* et de l'image chiffrée par substitution

On constate que le chiffrement par substitution modifie considérablement l'histogramme de l'image, rendant la distribution des niveaux de gris uniforme.

3.4 Déchiffrement de l'image

De la même manière que pour le chiffrement par permutation, j'ai écrit un programme `subst_inv.cpp` qui permet de déchiffrer une image chiffrée avec la même clé. Il s'appuie sur les opérations inverses du chiffrement par substitution. Là encore, aucune perte n'est introduite lors du déchiffrement.

3.5 Attaque par force brute

Après avoir implémenté le déchiffrement par substitution, j'ai réalisé une attaque par force brute sur cette méthode de chiffrement. Dans le cadre du TP, nous nous sommes échangé des images chiffrées par substitution au sein de la classe. L'objectif était de retrouver la clé utilisée pour chiffrer une image sans la connaître a priori.

Étant donné que la clé est codée sur 8 bits dans notre cas, il y a $2^8 = 256$ clés possibles. Plutôt que d'examiner manuellement les 256 images déchiffrées possibles, j'ai automatisé le processus en me basant sur le calcul de l'entropie de chaque image déchiffrée candidate.

L'attaque par force brute consiste à tester toutes les clés possibles pour déchiffrer l'image chiffrée. Pour chaque clé, j'ai procédé comme suit :

1. Déchiffrement de l'image avec la clé candidate
2. Calcul de l'entropie de l'image déchiffrée
3. Sélection de l'image ayant l'entropie la plus faible parmi les 256 images

L'hypothèse est que l'image correcte (c'est-à-dire l'image originale) aura une entropie plus faible que les images déchiffrées avec des clés incorrectes, car les images aléatoires tendent à avoir une entropie plus élevée.



Figure 6: Exemple d'image anciennement chiffrée crackée par force brute

Cette méthode s'est avérée très efficace sur toutes les images que j'ai pu tester. Elle met en lumière une faiblesse de cette méthode de chiffrement lorsque l'on utilise une clé de petite taille.

4 Conclusion

En conclusion, bien que les méthodes de chiffrement par permutation et substitution apportent un certain niveau de sécurité, elles présentent des vulnérabilités lorsqu'elles sont utilisées isolément et avec des clés de faible longueur. Pour renforcer la protection des images, on peut imaginer combiner ces deux techniques, ou utiliser des clés plus longues, voire utiliser des méthodes standardisées (AES, JPEG2000, ...). J'ai beaucoup aimé ce TP et son côté interactif, il m'a donné envie de découvrir plus en profondeur la sécurité multimédia.